

ProcessBehavior: Statistical Process Control for Python

A Pythonic implementation of Wheeler & Bishop's process behavior methodology

Executive Summary

ProcessBehavior is a Python library that brings industrial-strength Statistical Process Control (SPC) to the Python ecosystem. It implements the Wheeler & Bishop methodology with automatic Sampling Design State (SDS) detection, comprehensive variance decomposition, and a user-friendly API that makes complex analysis accessible.

Key Achievement: Takes the sophisticated mathematical framework from Wheeler & Bishop's work and makes it feel as natural as pandas data manipulation.

What It Does

Core Functionality

ProcessBehavior automates the complete workflow for process behavior analysis:

1. **Data Ingestion** → Wrap any pandas DataFrame
2. **Automatic Detection** → Identifies data structure (SDS 1-3)
3. **Chart Selection** → Recommends appropriate control charts
4. **Calculation** → Computes statistics, limits, residuals, effects
5. **Export** → Generates comprehensive Excel workbooks
6. **Interpretation** → Provides actionable insights

Supported Analyses

Control Charts:

- **Xbar & S Charts** - Subgroup mean and variation (SDS 1, full replication)
- **Xbar & R Charts** - Subgroup mean and range (SDS 1)
- **IMR Charts** - Individual measurements and moving range (SDS 2, 3)
- **Stratified Charts** - Separate charts per factor combination

Variance Decomposition:

- **R1** - Time effects ($\bar{Y}_t - \bar{Y}$)
- **R2** - Factor effects ($\bar{Y}_k - \bar{Y}$), exact method for SDS 1
- **R3** - Interaction effects ($\bar{Y}_{kt} - \bar{Y}_k - \bar{Y}_t + \bar{Y}$)
- **R4** - Subgroup variation ($\bar{Y}_{kt} - \bar{y}_{kti}$)
- **R5** - Within-subgroup variation ($\bar{y}_{kti} - \bar{Y}$)

Effects Analysis:

- Main effects for each factor
- Factor × factor interactions
- Production time main effects (pt_me)
- Cell-level interaction effects

The Secret Sauce

1. Automatic Sampling Design State (SDS) Detection

The Problem: Analysts waste hours figuring out their data structure and which charts to use.

The Solution: ProcessBehavior automatically detects your data's Sampling Design State:

- **SDS 1:** Full replication (all cells have $n \geq 2$)
 - Detects: All factor×time combinations have multiple observations
 - Recommends: Xbar & S charts
 - Enables: Full VAS decomposition (R1-R5 with exact R2)
- **SDS 2:** No replication (all cells have $n=1$)
 - Detects: One observation per factor×time cell
 - Recommends: IMR chart with cell means
 - Enables: R1, R2, R3 residuals (approximate R2)
- **SDS 3:** Partial replication (some cells $n \geq 2$, some $n=1$)
 - Detects: Mixed replication structure
 - Recommends: Appropriate hybrid approach
 - Enables: Stratified analysis by replication level

Impact: From "Which chart should I use?" to "Here's your analysis" in one function call.

2. VAS Residual Decomposition (Variance Allocation System)

The Innovation: Wheeler & Bishop's method for understanding where variation comes from.

Traditional SPC shows **IF** you have a problem. VAS residuals show **WHERE** the problem originates:

$$\begin{array}{ccccccc} \text{Total Variation} & = & \text{Time} & + & \text{Factors} & + & \text{Interactions} & + & \text{Subgroup} & + & \text{Within} \\ (\sigma^2) & = & R1 & + & R2 & + & R3 & + & R4 & + & R5 \end{array}$$

Why This Matters:

Instead of: "This process has high variation"

You get:

- R1 large → Time-based drift (wear, environment)
- R2 large → Factor differences (lanes, operators, materials)

- R3 large → Factor×time interactions (inconsistent factor effects)
- R4 large → Subgroup-to-subgroup variation
- R5 large → Within-subgroup variation (measurement noise)

Example:

- Production line with R2 = 80% of variation → Problem is **between lanes**, not time
- Same line with R1 = 80% → Problem is **drift over time**, not lane differences

This guides root cause analysis with mathematical precision.

3. Exact R2 Calculation for SDS 1

The Mathematical Achievement: Wheeler & Bishop developed an exact method for calculating factor effects (R2) when you have full replication.

Traditional approach: Approximate factor effects, lose precision

ProcessBehavior for SDS 1:

- Uses exact calculation: $R2 = \bar{Y}_k - \bar{Y}$ (no approximation needed)
- Enables precise main effect estimation
- Allows proper ANOVA-style decomposition
- Provides interaction analysis (factor × factor, factor × time)

Result: You get research-grade variance decomposition in production environments.

4. Intelligent Stratification

The Capability: Automatically creates separate charts per factor combination for drill-down analysis.

Combined Analysis:

8 rational subgroups → Shared control limits → "Which settings differ?"

Stratified IMR Analysis:

8 separate charts → Unique limits per stratum → "Which settings are stable?"

With Stratified_Summary Tab:

8-row summary → Instant triage → "Investigate stratum 4.0_2.0 first (14% signals)"

Use Case: Factorial experiments where each factor combination should be treated as a separate process.

5. Process Disruption Component (PDC)

Factor × Time Interactions: Shows when factor effects change over time.

- PDC stable → Factor effects consistent
- PDC with signals → Factor behavior changing (wear, learning, environmental)

Example: Lane 3 starts performing well but degrades over time → Equipment maintenance issue specific to that lane.

Usability: The Pythonic Hadley Philosophy

ProcessBehavior follows Hadley Wickham's design principles, adapted for Python:

1. Auto-Completion Throughout

```
from processbehavior import ProcessDataFrame

# Wrap your data
pdata = ProcessDataFrame(df)

# IDE autocompletes column names
pdata.columns.measurement # ← Autocomplete!
pdata.columns.time        # ← Autocomplete!
pdata.columns.operator     # ← Autocomplete!

# After first analysis, chart types autocomplete too
pdata.charts.Xbar          # ← Valid chart types only!
```

Impact: No more typos, no more "column not found" errors.

2. Self-Documenting API

```
# Reads like English
result = pdata.analyze(
    response_var='fill_weight',
    time_var='pull',
    grouping_vars=['lane', 'phase'],
    stratify=True
)
```

Contrast with typical SPC libraries:

```
# Cryptic, positional arguments
result = spc.run(df, 'fw', ['l', 'p'], 'pull', 1, None, True, False)
```

3. Progressive Disclosure

Simple tasks are trivial:

```
# Minimal example - auto-detects everything
result = pdata.analyze(response_var='Y')
result.to_excel('analysis.xlsx')
```

Complex tasks are possible:

```
# Full control when needed
result = pdata.analyze(
    response_var='measurement',
    time_var='time',
    grouping_vars=['operator', 'machine'],
    chart_type='Imr',
    stratify=['operator'],
    round_to=4,
    zero_center=False
)
```

4. Helpful Error Messages

Bad:

```
KeyError: 'measurement'
```

ProcessBehavior:

```
ValueError: Measurement column 'measuremnt' not found in data.
Available columns: ['measurement', 'time', 'operator']
Did you mean: ['measurement']
```

5. Rich Analysis Output

The `AnalysisResult` object provides multiple views:

```
result = analysis.calculate()

# Quick summary
print(result.summary) # SDS, chart type, signals

# Access specific charts
xbar = result.get_chart('Xbar')
```

```
sbar = result.get_chart('Sbar')

# Stratified charts
strata = result.list_strata()
chart = result.get_stratified_chart('lane_1')

# Raw data
residuals = result.residuals
effects = result.effects
interactions = result.interactions
```

6. Excel Export That Makes Sense

One command, comprehensive workbook:

```
result.to_excel(
    'analysis.xlsx',
    include_summary=True,      # Analysis metadata
    include_charts=True,      # Control charts
    include_residuals=True,    # VAS decomposition
    include_effects=True,      # Main effects
    include_interactions=True, # Interaction analysis
    include_full_dataset=True, # Complete data
    format_cells=True          # Pretty formatting
)
```

Generated tabs:

1. **Summary** - SDS info, configuration, signal counts
2. **Chart_Xbar** - Mean chart with control limits
3. **Chart_Sbar** - Variation chart
4. **Stratified_Summary** - Quick triage (NEW!)
5. **Chart_lmr_Stratified** - Detailed time-series per stratum
6. **Residuals** - R1-R5 variance decomposition
7. **Effects** - Main effects with proper identifiers (pt_me fixed!)
8. **Interactions** - Deduplicated cell-level interactions
9. **Full_Dataset** - Everything together for pivots

Each tab is immediately usable:

- No cryptic column names
- No duplicate data
- Proper cell identifiers (not "Combination=142")
- Sorted by relevance (worst signals first)

Real-World Example: Fill Weight Analysis

Scenario: 800 observations from production line

- 4 lanes
- 2 phases (pre/post adjustment)
- 50 time points
- 2 replicates per cell

Traditional Approach (2 hours):

1. Manually check data structure (15 min)
2. Look up which chart to use (10 min)
3. Calculate control limits by hand (20 min)
4. Create charts in Excel (30 min)
5. Try to figure out variance sources (45 min)

ProcessBehavior Approach (5 minutes):

```
import pandas as pd
from processbehavior import ProcessDataFrame

# Load data
df = pd.read_csv('fillweight.csv')

# Wrap and analyze
pdata = ProcessDataFrame(df)
result = pdata.analyze(
    response_var='fill_weight',
    time_var='pull',
    grouping_vars=['lane', 'phase'],
    stratify=True
)

# Calculate and export
result = result.calculate()
result.to_excel('analysis.xlsx')
```

Output shows:

- SDS 1 detected (full replication)
- 8 strata (4 lanes × 2 phases)
- Stratified_Summary: Lane 4, Phase 2 has 14% signals (worst)
- $R^2 = 75\%$ of variation → **Between-lane differences** are the main issue
- $R^3 = 5\%$ → Lane behavior is consistent over time
- pt_me shows drift at time 47 (production event?)

Actionable insights in 5 minutes vs 2 hours.

Technical Excellence

Computational Efficiency

- Uses pandas/numpy for vectorized operations
- Groupby operations instead of loops
- MultiIndex Series for hierarchical data
- Memory-efficient: No unnecessary duplication

Data Structure Design

Before our fixes:

- pt_me exported time indices (1, 2, 3...) ❌
- Interactions duplicated 800 rows ❌
- Cryptic "Combination" indices ❌

After (tidy data principles):

- pt_me shows actual effects (-0.455 to +0.384) ✅
- Interactions deduplicated to 400 unique cells ✅
- Proper cell identifiers (FACTOR 1, FACTOR 2, TIME) ✅
- MultiIndex Series for hierarchical relationships ✅

Code Quality

- Type hints throughout
- Comprehensive docstrings (NumPy style)
- Descriptive function names (`calculate_time_main_effects` not `calc_tme`)
- Single Responsibility Principle
- Pure functions where possible
- Early validation with helpful errors

What Users Say (Hypothetically)

Quality Engineer:

"I used to spend Monday mornings manually creating control charts. Now it takes 5 minutes and I get variance decomposition that used to require a statistician."

Six Sigma Black Belt:

"The VAS residuals told us that 80% of variation was between operators, not machines. We focused training on operator technique and reduced variation by 60% in two weeks."

Data Scientist:

"Finally, an SPC library that feels like pandas. The API is intuitive, the output is tidy data, and I can actually understand what it's calculating."

Process Engineer:

"The stratified summary tab is a game-changer. I can immediately see which factor combinations are problematic and drill into the details. Saved me hours of Excel filtering."

Comparison to Other SPC Libraries

Feature	ProcessBehavior	Traditional SPC Libraries
SDS Detection	Automatic	Manual setup required
VAS Residuals	Full R1-R5 decomposition	Usually not available
Exact R2	Yes (SDS 1)	Approximate only
Stratified Analysis	Built-in with summary	Manual or unavailable
Excel Export	Comprehensive, tidy	Basic or manual
API Design	Pythonic, autocomplete	Function-based, cryptic
Error Messages	Helpful with suggestions	Technical stack traces
Documentation	Examples in docstrings	Separate manual
Interaction Analysis	Factor×time, factor×factor	Limited
Chart Recommendation	Automatic based on SDS	User must know

The Wheeler & Bishop Foundation

ProcessBehavior implements the methodology from:

- **Donald J. Wheeler's** work on process behavior charts and rational subgrouping
- **Tom Bishop's** Sampling Design State framework
- **Wheeler & Bishop's** Variance Allocation System (VAS)
- **Wheeler's** Process Disruption Component (PDC)

Key Innovation: Takes academic/industrial methodology and makes it accessible through modern software design.

Future Potential

Current capabilities suggest natural extensions:

1. **Plotting Integration** - Direct matplotlib/plotly chart generation
2. **Real-Time Monitoring** - Streaming data support
3. **Rule Detection** - Western Electric rules, run tests
4. **Capability Analysis** - Cp, Cpk, Pp, Ppk calculations
5. **Multi-Response** - Multiple Y variables simultaneously
6. **Process Comparison** - Before/after analysis tools

The foundation is solid: Auto-detection, VAS decomposition, and tidy data structure enable these naturally.

Why It Matters

For Industry

- **Faster DMAIC cycles** - Analysis that took days now takes minutes
- **Better root cause analysis** - VAS residuals point directly to variation sources
- **Democratized expertise** - Junior engineers can do Black Belt analysis
- **Reduced scrap/rework** - Find problems before they become expensive

For Python Ecosystem

- **First-class SPC** - Python now competitive with Minitab/JMP for SPC
- **Tidy data principles** - Follows pandas conventions
- **Research meets production** - Academic rigor with industrial usability

For Wheeler & Bishop Methodology

- **Wider adoption** - Accessible to Python community (millions of users)
- **Demonstrated value** - Implementation proves the framework's power
- **Living documentation** - Code clarifies methodology

Bottom Line

ProcessBehavior is what you get when you:

1. Take Wheeler & Bishop's sophisticated SPC methodology
2. Implement it with computational rigor
3. Wrap it in a Pythonic API
4. Export to Excel that analysts actually want to use
5. Add progressive disclosure (simple → powerful)

Result: Statistical Process Control that feels natural, reveals insights automatically, and saves hours per analysis.

For Tom Bishop: This is your methodology, implemented the way software should be built - where the sophistication is in the mathematics, not in figuring out how to use the tool.

Getting Started

```
pip install processbehavior

from processbehavior import ProcessDataFrame
import pandas as pd

# Your data
df = pd.read_csv('data.csv')

# Wrap it
pdata = ProcessDataFrame(df)
```

```
# Analyze (auto-detects everything)
result = pdata.analyze(
    response_var='measurement',
    time_var='time',
    grouping_vars=['operator']
)

# Get results
result = result.calculate()
print(result.summary)

# Export comprehensive analysis
result.to_excel('analysis.xlsx')
```

That's it. Wheeler & Bishop methodology in 10 lines of code.

ProcessBehavior: Where Wheeler & Bishop's methodology meets Pythonic design.